MIT 6.006

13 October 2026 12:35

Lecture 1: Algorithms & Computations

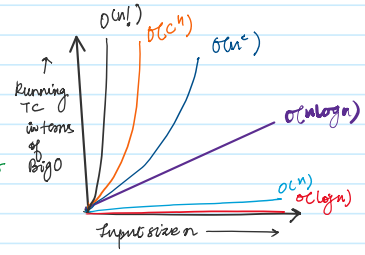
Goal → 1) solve computational problems
2) prove correctness
3) argue efficiency
4) communication of problems

Problem: possible inputs & outputs
↳ computational problem: binary relation b/w set of inputs & outputs

Correctness: use Induction to prove correctness in recursion questions
Base Case: $n=1$ or $n=0$
Inductive hypothesis: assume the statement is true for some arbitrary natural number
Inductive step: using the hypothesis assumption, prove it's true for $k+1$

Efficiency
By counting fundamental operations → also referred to as Frequency count method.
use Asymptotic analysis that helps measure ops
→ Don't measure time but operation
→ expect performance to depend on size of input
Types: 1) Big O (upper bound)
2) Big Omega Ω (lower bound)
3) Theta (Θ) - (both-average)

* Refer to Module Bani notes on TC + SC



How to map with computer's memory?

RAM (Random access memory) – we assume computer can access any address in memory in constant time
memory is Byte addressable
Memory is conceptually → an arrangement of addresses



↳ 8 bits = 1 byte
1 bit is 0 or 1

O(1) O(logn) O(n) O(nlogn) O(n^2) O(n^ex) O(2^n)

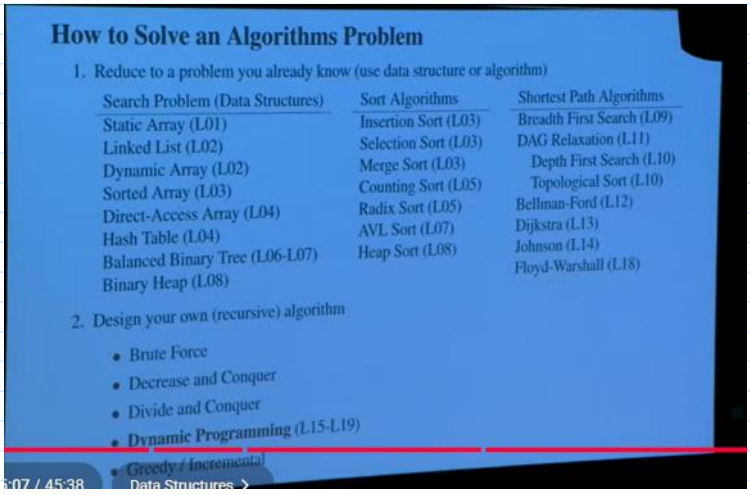
What about 64 bits in computers?
that means CPU has registers that can hold 64 bits = 8 bytes × 8 bits



$= 2^n$ distinct addresses
$= 2^{64}$

64 bit computer can have gigabytes of RAM with billions bytes
64 describe the width of certain processors operations, registers and address-related capabilities

↳ RAM capacity → storage available to hold data
memory address → a bit within the addressable space
64 bit register → inside CPU for operations
64 bit word → natural data unit → CPU's natural processing width
word size → unit of data that processor can handle in a single operation
= native data size of the architecture = 64 bits



Lecture 2: Data structure & Dynamic arrays

Three fundamental concepts:
1) Abstract data type (ADT)
what operations are available? Behaviour of data type
The contract in stack:
push(), pop(), peek(), LIFO order
↓ standford course focus on them
2) Data structure (Representation)
How is data organized & stored to support those operations?
eg: stack can be implemented using dynamic array or a linked list
↓ linear / non linear — how elements r organized
3) Algorithm
what sequence of computational steps support those operations?

2 main interfaces: + 2 main data structure approaches
1) Sequence 2) Set 1) Arrays 2) Pointers.
↳ 1) static sequence interface

1) Sequence 2) Set 1) Arrays 2) Pointers.

↳ 1) Static sequence interface

sequence of items $x_0, x_1, \dots x_{n-1}$

build(x): O(n)

len(): O(1) → python stores size of collection in the memory by default

iter_seq(): O(n)

get_at(i): O(1)
set_at(i, x): O(1)
} as array addressing makes it easy to access + word size is $\log n$ at least

which is [static arrays]

python has no static array

memory:

considering we have "word RAM" model

array is w-bit words + consecutive chunk

1 word size = $\log n$ at least why?

as to perform get at & set at in constant time

machine should be able to represent any index i

within single machine word

So the word size in modern computer = 64 bits ~ $\log_2(6)$



array[i] = memory[address(0) + i]

hence make access O(1)

↳ 2) Dynamic Sequence:

[static sequence] +

1) insert_at(i, x) → shift all items post (next element) i^th position } O(n)

2) delete_at(i) → shift all to fill the space

3) insert/delete at first or last O(1) for last as space always easier in python — else O(1) for static array

O(n) for first as data needs to shift

Dynamic Seq operations:

Static Array

1) insert and delete cost O(n) time

→ at last it's O(1) in python else O(n) in all cases

Linked list

Insert & delete at front at O(1)

[x] just need head address change

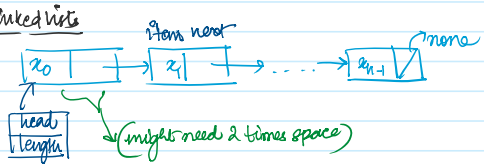
else O(n)

get + set → O(i) time or O(n) as each node should be checked

Bad at random access as the position needs to be travelled every time

For last access store [head | len | tail] Data Structure Augmentation – Add new info to make things better

Linked lists

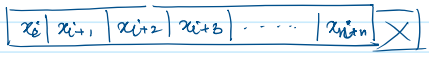


None of the above work efficiently, hence we

Solution → Dynamic array (Python lists)

→ Relaxes constraints on the size(array) = n (Roughly n)

→ size of array $\Theta(n)$ & $\geq n$ ⟶ if we visualize :



Array
length = n (of items i stored)
size = (of the data structure Array with empty spaces)

→ maintain $A[i] = x_i$

① So assume we keep buffer & then extend it as the data grows!!

② But what if n = size?

allocate a new array of size = 2 × size

how will that help?

Eg: We do insert_last() from empty array

pass 1: [1] increase twice so size = 2

[1 | 2 | |] size = 4

& so on size = 1, 2, 4, 8, 16

} → resize cost = $\Theta(1+2+4+8+16+\dots+n)$

$= \Theta\left(\sum_{i=1}^{\log n} 2^i\right) = \Theta(2^{\log n}) = \Theta(n)$ ≈ Amortized O(1)



Size = 4

& so on size = 1, 2, 4, 8, 16
(2^0 for nothing)

→ resize cost = $\theta(1+2+4+8+16+\dots+n)$
$= \theta\left(\sum_{i=1}^{\log n} 2^i\right) = \theta(2^{\log n}) = \theta(n)$ ≈ Amortized $\theta(1)$

in geometric sequence, where seq is growing exponentially, we can simply pick the last item

Amortization

Operation takes $T(n)$ amortized time if any k operations take $\leq kT(n)$ time

or simply →
"averaging over operation sequence" ← Total time it takes to do n operations cannot take more than n time — over the long term, this is on the average

Sequence Interface

Data Structures	Static	Dynamic		
	get_at(i) set_at(i,x)	insert_first(x) delete_first()	insert_last(x) delete_last()	insert_at(i,x) delete_at(i)
Array	1	n	n	n
Linked list	n	1	n	n
dynamic array	1	n	1 (a)	n

Problem for Lec1 (Introduction to Algorithms – Problem session 1 – Asymptotic Behavior of Functions & Double ended)

$\sqrt{n} < n < n+\sqrt{n}$

$\log(n) = o(n^b)$

How big is $n!$
→ Stirling's approximation
$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

so, $\log(n!) = \log\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) \to \theta(n\log n)$

Binomial coefficient

$\binom{n}{k} = \frac{n!}{k!(n-k)!}$

no of n ways to do k

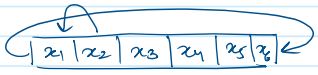
Implementation of an interface is array, linked list etc

Manipulating a sequence via Black-Box operations
Q. Swap_ends (Data structure) using data structure specific operations

swap_ends(arr):
x_1 = arr.delete_first()
x_2 = arr.delete_last()
arr.insert_first(x_2)
arr.insert_last(x_1)

(you could use temp and simply swap)

Q. shift left
shift_left(...) in $O(k)$



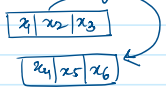
put x_1 behind x_6 and x_2 to x_1

Brute force
delete x_1 & insert at x_6 for n times?
x_2 is naturally at x_1 now?

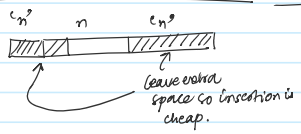
Recursive?

if k=0 or k==1:
return
x = delete_first()
insert_last(x)
shift_left(0, k-1)
↖ array indices

let's say shift from x_3



So let's split the array & reverse the first half and stitch it at the end?



Q Efficient data structure – Dynamic arrays in python

Dynamic array has doubly ended space.
What about extra space? – At no point we should have extra space & waste it
"Keep some space always"

Q. Given singly linked list of size 2n, reverse the order of the last n nodes without using extra constant space

Steps:
1) find the nth element
2) iterate & assign the next node pointer to previous one
3) Refine the end of first half to the new head.

1) find the nth element
2) iterate & assign the next node pointer to previous one
3) Reline the end of first half to the new head.

TODO: implement this

Lecture 3: sets and sorting

Sets - maintains order in keys

interface:

Container	build(A)	↳ given an iterable, build sequence using them
	len()	
static	find(k)	
dynamic	insert(x)	add x, and if already exist, no
	delete(k)	
Order	iter_ord()	return in key order
	find_min(), find_max(), find_next(k), find_prev(k)	

Implementation:
↳ Unordered array would always result in O(n) search time
↳ what if we do the work beforehand & sort?

	Container	Static	Dynamic	Order	
Data Structure	build(A)	find(k)	insert(x) / delete(k)	find_min() / find_max()	find_prev(k) / find_next(k)
Arrays	n	n	n	n	n
Linked list	nlogn	logn	n	1	logn

↳ [Input → array of len A; output → sorted array of len A]

Sorting vocabulary
1) Destructive - overwrites the input array
2) In place - uses O(1) extra space

Sorting Algorithms:
1) Permutation sort:

```
def permutation_sort(A):
    for B in permutation(A):
        if B_sorted(B):
            return B
```

① enumerate array A Ω(n!)
② check if that permutation is sorted?

```
for i in range(0, n)
    if arr[i] ≤ arr[i+1]:   O(n)
```

Ω(n!.n)
↳ lower bound as this is an assumption

2) selection sort
find biggest element and swap with element at last.